Data Structures - Homework 4

Q1. Implement a Stack using Queue data structure.

Code:

```
#include <stdio.h>
#define MAX 100
int queue[MAX];
int front = 0, rear = -1;

void enqueue(int x)
{
    queue[++rear] = x;
}

int dequeue()
{
    return queue[front++];
}

void push(int x)
{
    int i, n;
    n = rear - front + 1;
    enqueue(x);
    for (i = 0; i < n; i++)
    {
        enqueue(dequeue());
    }
}

int pop()
{
    if (front > rear)
    {
        printf("Stack Underflow\n");
        return -1;
    }
    return dequeue();
}

void display()
{
    int i;
    if (front > rear)
    {
        printf("Stack is Empty\n");
        return;
    }
    printf("Stack: ");
    for (i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main()
```

```
{
    int choice, x;
    while (1)
    {
        printf("\n1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter element: ");
                scanf("%d", &x);
                push(x);
                break;
            case 2:
                x = pop();
                if (x != -1)
                    printf("Popped element: %d\n", x);
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

Output:

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 30
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped element: 30
1. Push
2. Pop
```

```
3. Display
4. Exit
Enter your choice: 3
Stack: 20 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
=== Code Execution Successful ===
```

Q2. Implement a Queue using Stack data structure.

Code:

```
#include <stdio.h>
#define MAX 100
int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;

void push1(int x)
{
    s1[++top1] = x;
}

int pop1()
{
    return s1[top1--];
}

void push2(int x)
{
    s2[++top2] = x;
}

int pop2()
{
    return s2[top2--];
}

void enqueue(int x)
{
    if (top1 == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }
    push1(x);
}

int dequeue()
{
    int x;
    if (top1 == -1 && top2 == -1)
    {
        printf("Queue Underflow\n");
        return -1;
    }
    if (top2 == -1)
    {
        while (top1 != -1)
```

```
        {
            x = pop1();
            push2(x);
        }
    }
    return pop2();
}

void display()
{
    int i;
    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is Empty\n");
        return;
    }
    printf("Queue: ");
    for (i = top2; i >= 0; i--)
        printf("%d ", s2[i]);
    for (i = 0; i <= top1; i++)
        printf("%d ", s1[i]);
    printf("\n");
}

int main()
{
    int choice, x;
    while (1)
    {
        printf("\n1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter element: ");
                scanf("%d", &x);
                enqueue(x);
                break;
            case 2:
                x = dequeue();
                if (x != -1)
                    printf("Dequeued element: %d\n", x);
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

Output:

```
1. Enqueue
```

```
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 20
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue: 10 20 30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued element: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue: 20 30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
=== Code Execution Successful ===
```

Q3. Write a C program using stack data structure to implement managing the history of web pages visited.

Code:

```
#include <stdio.h>
#include <string.h>
#define MAX 100
char history[MAX][100];
int top = -1;

void visit(char page[])
{
    if (top == MAX - 1)
    {
        printf("History is Full\n");
        return;
    }
```

```
    top++;
    strcpy(history[top], page);
    printf("Visited: %s\n", page);
}

void back()
{
    if (top == -1)
    {
        printf("No previous page\n");
        return;
    }
    printf("Going back from: %s\n", history[top]);
    top--;
    if (top != -1)
        printf("Current page: %s\n", history[top]);
    else
        printf("No page in history\n");
}

void display()
{
    int i;
    if (top == -1)
    {
        printf("History is Empty\n");
        return;
    }
    printf("\nWeb Page History:\n");
    for (i = top; i >= 0; i--)
    {
        printf("%s\n", history[i]);
    }
}

int main()
{
    int choice;
    char page[100];
    while (1)
    {
        printf("\n1. Visit Page\n");
        printf("2. Back\n");
        printf("3. Display History\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter web page: ");
                scanf("%s", page);
                visit(page);
                break;
            case 2:
                back();
                break;
            case 3:
                display();
                break;
            case 4:
```

```
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

Output:

```
1. Visit Page
2. Back
3. Display History
4. Exit
Enter your choice: 1
Enter web page: GOOGLE
Visited: GOOGLE
1. Visit Page
2. Back
3. Display History
4. Exit
Enter your choice: 1
Enter web page: insta
Visited: insta
1. Visit Page
2. Back
3. Display History
4. Exit
Enter your choice: 1
Enter web page: yt
Visited: yt
1. Visit Page
2. Back
3. Display History
4. Exit
Enter your choice: 2
Going back from: yt
Current page: insta
1. Visit Page
2. Back
3. Display History
4. Exit
Enter your choice: 3
Web Page History:
insta
GOOGLE
1. Visit Page
2. Back
3. Display History
4. Exit
Enter your choice: 4
=== Code Execution Successful ===
```

Q4. Write a C program to sort a stack using temporary stack (implement using stack).

Code:

```
#include <stdio.h>
#define MAX 100
int stack[MAX], temp[MAX];
```

```
int top = -1, ttop = -1;

void push(int x)
{
    stack[++top] = x;
}

int pop()
{
    return stack[top--];
}

void tpush(int x)
{
    temp[++ttop] = x;
}

int tpop()
{
    return temp[ttop--];
}

void sortStack()
{
    int x;
    while (top != -1)
    {
        x = pop();
        while (ttop != -1 && temp[ttop] > x)
        {
            push(tpop());
        }
        tpush(x);
    }
    while (ttop != -1)
    {
        push(tpop());
    }
}

void display()
{
    int i;
    printf("Stack: ");
    for (i = top; i >= 0; i--)
    {
        printf("%d ", stack[i]);
    }
    printf("\n");
}

int main()
{
    int n, i, x;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &x);
        push(x);
```

```
    }
    printf("\nOriginal Stack: ");
    display();
    sortStack();
    printf("Sorted Stack: ");
    display();
    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements:
12
23
34
45
6
Original Stack: Stack: 6 45 34 23 12
Sorted Stack: Stack: 6 12 23 34 45
=== Code Execution Successful ===
```

Q5. The Stock Span Problem. Given daily price quotes for a stock, calculate the span of the stock's price for all days. The span for day i is the maximum number of consecutive days leading up to i (including i) where the price is less than or equal to the price on day i.

Input: [100, 80, 60, 70, 60, 75, 85] Output: [1, 1, 1, 2, 1, 4, 6]

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int price[MAX], span[MAX];
    int stack[MAX];
    int top = -1;
    int n, i;
    printf("Enter number of days: ");
    scanf("%d", &n);
    printf("Enter stock prices:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &price[i]);
    }
    for (i = 0; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
        {
            top--;
        }
        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];
        stack[++top] = i;
    }
    printf("Stock Span: ");
    for (i = 0; i < n; i++)
    {
```

```
            printf("%d ", span[i]);
        }
        return 0;
    }
```

Output:

```
Enter number of days: 7
Enter stock prices:
100
80
60
70
60
75
85
Stock Span: 1 1 1 2 1 4 6
=== Code Execution Successful ===
```

Q6. Given an array, for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists, set value to -1. Implement using stack.

Input: [2, 1, 1, 3, 2, 1] Output: [1, -1, -1, 2, 1, -1]

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int arr[MAX], freq[MAX];
    int stack[MAX], result[MAX];
    int n, i, j;
    int top = -1;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    for (i = 0; i < n; i++)
    {
        freq[i] = 0;
        for (j = 0; j < n; j++)
        {
            if (arr[i] == arr[j])
            {
                freq[i]++;
            }
        }
    }
    for (i = n - 1; i >= 0; i--)
    {
        while (top != -1 && freq[stack[top]] <= freq[i])
        {
            top--;
        }
        if (top == -1)
            result[i] = -1;
```

```
            else
                result[i] = arr[stack[top]];
            stack[++top] = i;
        }
        printf("Output: ");
        for (i = 0; i < n; i++)
        {
            printf("%d ", result[i]);
        }
        return 0;
    }
```

Output:

```
Enter number of elements: 6
Enter elements:
2
1
1
3
2
1
Output: 1 -1 -1 2 1 -1
=== Code Execution Successful ===
```

Q7. Given a pattern containing only 'I's and 'D's ('I' for Increasing, 'D' for Decreasing), devise an algorithm using stack to print the minimum number following that pattern. Digits from 1-9 and digits can't repeat.

Input: D -> Output: 21 | Input: I -> Output: 12 | Input: IDID -> Output: 21435

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
    char pattern[MAX];
    int stack[MAX];
    int top = -1;
    int num = 1;
    int i;
    printf("Enter pattern: ");
    scanf("%s", pattern);
    printf("Output: ");
    for (i = 0; pattern[i] != '\0'; i++)
    {
        stack[++top] = num++;
        if (pattern[i] == 'I')
        {
            while (top != -1)
            {
                printf("%d", stack[top--]);
            }
        }
    }
    stack[++top] = num++;
    while (top != -1)
    {
        printf("%d", stack[top--]);
    }
```

```
    return 0;
}
```

Output:

```
Enter pattern: D
Output: 21
=== Code Execution Successful ===
```

Q8. Given an array of digits (0 to 9), find the minimum possible sum of two numbers formed from the digits of the array. All digits must be used to form the two numbers. Use Queue for implementation.

Input: [6, 8, 4, 5, 2, 3] Output: 604 (min sum formed by numbers 358 and 246)

Code:

```
#include <stdio.h>
#define MAX 100
int queue[MAX];
int front = 0, rear = -1;

void enqueue(int x)
{
    queue[++rear] = x;
}

int dequeue()
{
    return queue[front++];
}

int main()
{
    int n, i, j, temp;
    int num1 = 0, num2 = 0;
    printf("Enter number of digits: ");
    scanf("%d", &n);
    printf("Enter digits:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &queue[i]);
    }
    rear = n - 1;
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (queue[i] > queue[j])
            {
                temp = queue[i];
                queue[i] = queue[j];
                queue[j] = temp;
            }
        }
    }
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
            num1 = num1 * 10 + dequeue();
        else
```

```
            num2 = num2 * 10 + dequeue();
        }
        printf("Minimum Sum = %d\n", num1 + num2);
        return 0;
    }
```

Output:

```
Enter number of digits: 6
Enter digits:
6
8
4
5
2
3
Minimum Sum = 604
=== Code Execution Successful ===
```

Q9. Given an array and a positive integer k, find the first negative integer for each window (contiguous subarray) of size k. If a window has no negative integer, print 0 for that window. Implement using queue.

Input: [-8, 2, 3, -6, 10], k = 2 Output: [-8, 0, -6, -6]

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int arr[MAX], queue[MAX];
    int front = 0, rear = -1;
    int n, k, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    printf("Enter window size: ");
    scanf("%d", &k);
    printf("Output: ");
    for (i = 0; i < n; i++)
    {
        if (arr[i] < 0)
            queue[++rear] = i;
        if (front <= rear && queue[front] <= i - k)
            front++;
        if (i >= k - 1)
        {
            if (front <= rear)
                printf("%d ", arr[queue[front]]);
            else
                printf("0 ");
        }
    }
    return 0;
}
```

Output:

```
Enter number of elements: 5
```

```
Enter elements:
-8
2
3
-6
10
Enter window size: 2
Output: -8 0 -6 -6
=== Code Execution Successful ===
```

Q10. Given a matrix of dimension m x n where each cell can be 0 (empty), 1 (fresh orange), or 2 (rotten orange), find the minimum time required so that all oranges become rotten. A rotten orange at (i, j) can rot its neighbours (up, down, left, right). If impossible, return -1. Use Queue for implementation.

Input: [[2,1,0,2,1],[1,0,1,2,1],[1,0,0,2,1]] Output: 2

Code:

```
#include <stdio.h>
#define MAX 100
struct Node
{
    int row;
    int col;
    int time;
};
int main()
{
    int a[MAX][MAX];
    struct Node queue[MAX * MAX];
    int front = 0, rear = -1;
    int r, c, i, j;
    int fresh = 0;
    int time = 0;
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);
    printf("Enter matrix:\n");
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
            if (a[i][j] == 1)
                fresh++;
            if (a[i][j] == 2)
            {
                rear++;
                queue[rear].row = i;
                queue[rear].col = j;
                queue[rear].time = 0;
            }
        }
    }
    while (front <= rear)
    {
        struct Node current = queue[front++];
        int x = current.row;
        int y = current.col;
        int t = current.time;
```

```
            if (t > time)
                time = t;
            if (x > 0 && a[x-1][y] == 1)
            {
                a[x-1][y] = 2;
                fresh--;
                rear++;
                queue[rear].row = x-1;
                queue[rear].col = y;
                queue[rear].time = t + 1;
            }
            if (x < r-1 && a[x+1][y] == 1)
            {
                a[x+1][y] = 2;
                fresh--;
                rear++;
                queue[rear].row = x+1;
                queue[rear].col = y;
                queue[rear].time = t + 1;
            }
            if (y > 0 && a[x][y-1] == 1)
            {
                a[x][y-1] = 2;
                fresh--;
                rear++;
                queue[rear].row = x;
                queue[rear].col = y-1;
                queue[rear].time = t + 1;
            }
            if (y < c-1 && a[x][y+1] == 1)
            {
                a[x][y+1] = 2;
                fresh--;
                rear++;
                queue[rear].row = x;
                queue[rear].col = y+1;
                queue[rear].time = t + 1;
            }
        }
        if (fresh > 0)
            printf("Minimum Time = -1\n");
        else
            printf("Minimum Time = %d\n", time);
        return 0;
    }
```

Output:

```
Enter rows and columns: 3 5
Enter matrix:
2
1
0
2
1
1
0
1
2
1
1
0
```

```
0
2
1
Minimum Time = 2
=== Code Execution Successful ===
```